南京大学
NANJING UNIVERSITY

数据结构
Data Structure

许 蕾

xlei@nju.edu.cn

南京大学计算机学院

第三章 栈和队列

栈
队列
优先级队列

} **线性结构**

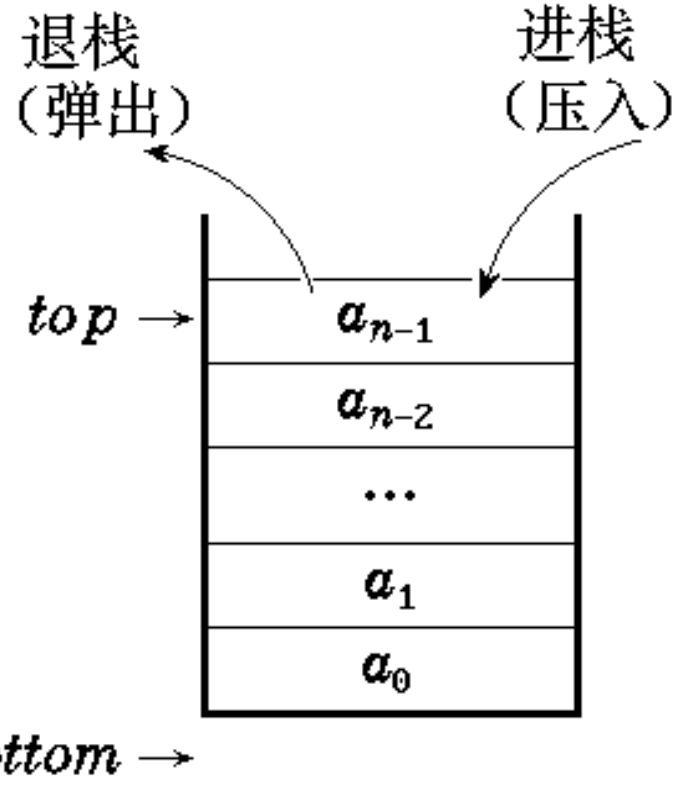
与表不同的是，它们都是限制存取位置的线性结构

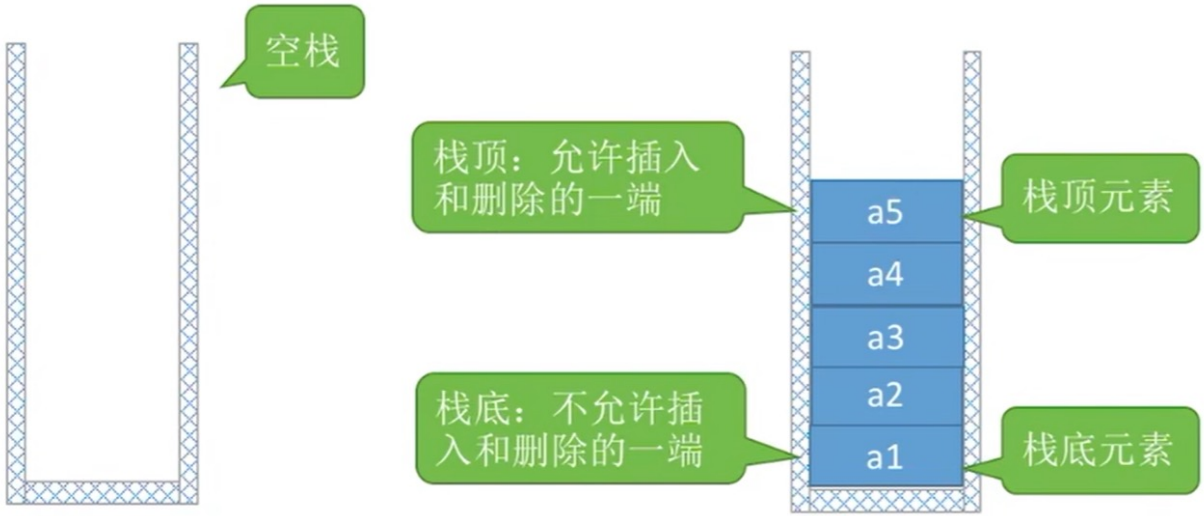
§3.1 栈（stack）

3.1.1 栈的定义

- 只允许在一端插入和删除的线性表
- 允许插入和删除的一端称为**栈顶**(*top*)，另一端称为**栈底**(*bottom*)
- 特点

后进先出(*LIFO*)





进栈顺序：
a1 → a2 → a3 → a4 → a5

出栈顺序：
a5 → a4 → a3 → a2 → a1

特点：后进先出
Last In First Out （LIFO）

1、一个栈的入栈序列是a，b，c，d，e，则下列哪个是不可能的输出序列？_____。

A. edcba　B. decba　C. dceab　D. abcde

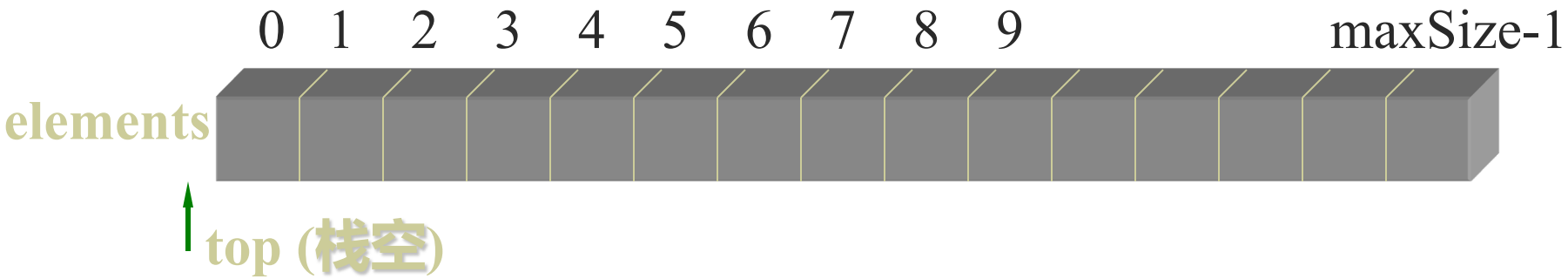
2、若栈的输入序列是1，2，3，…，n，输出序列的第一个元素是n，则第i个（i>=1）输出的元素是_____。

A. n-i　B. n-i+1　C. n-i-1　D. i

栈的抽象数据类型

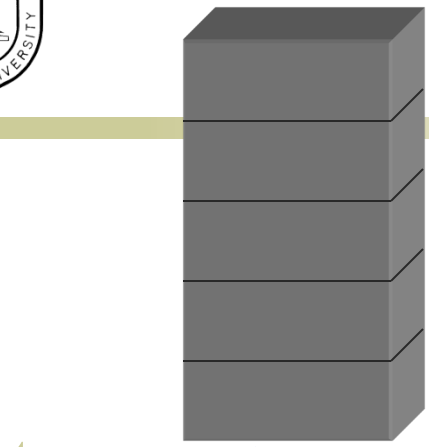
```
template <class E> class Stack {
public:
    Stack ( int=10 );                    //构造函数
    void Push ( const E & item);         //进栈
    E Pop ( );                           //出栈
    E getTop ( );                        //取栈顶元素
    void makeEmpty ( );                  //置空栈
    int IsEmpty ( ) const;               //判栈空否
    int IsFull ( ) const;                //判栈满否
}
```

3.1.2 顺序栈（栈的数组表示）

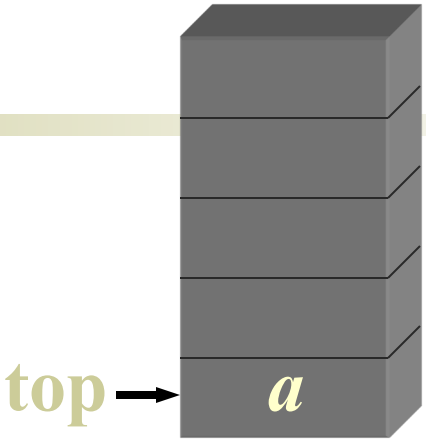



```
#include <assert.h>
template <class E>
class SeqStack : public Stack<E> {
private:
    int top;                        //栈顶指针
    E *elements;                    //栈元素数组
    int maxSize;                    //栈最大容量
    void overflowProcess();         //栈的溢出处理
```

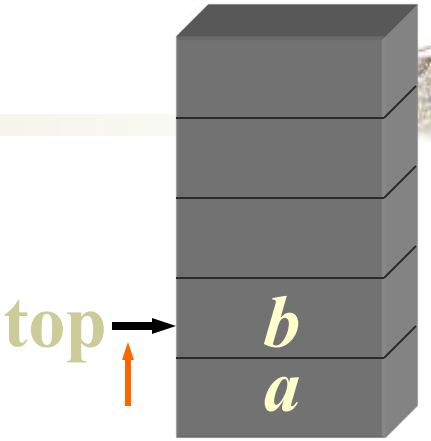
```
public:
   Stack (int sz = 10);              //构造函数
   ~Stack ( ) { delete [ ] elements; }
   void Push (E x);                  //进栈
   int Pop (E& x);                   //出栈
   int getTop (E& x);                //取栈顶
   void makeEmpty ( ) { top = -1; }  //置空栈
   int IsEmpty ( ) const { return top == -1; }
   int IsFull ( ) const
      { return top == maxSize-1; }
}
```



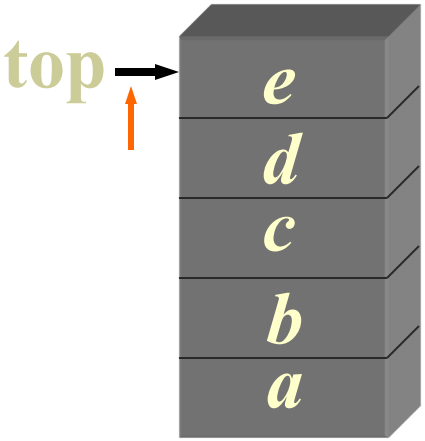
空栈



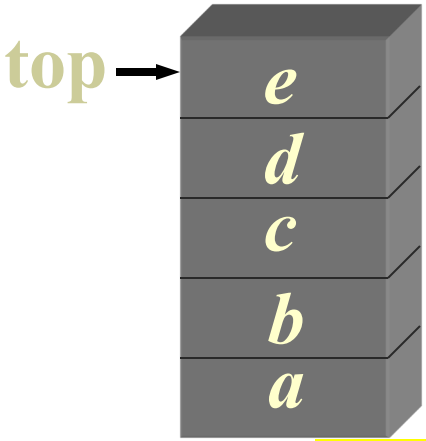
a 进栈



b 进栈

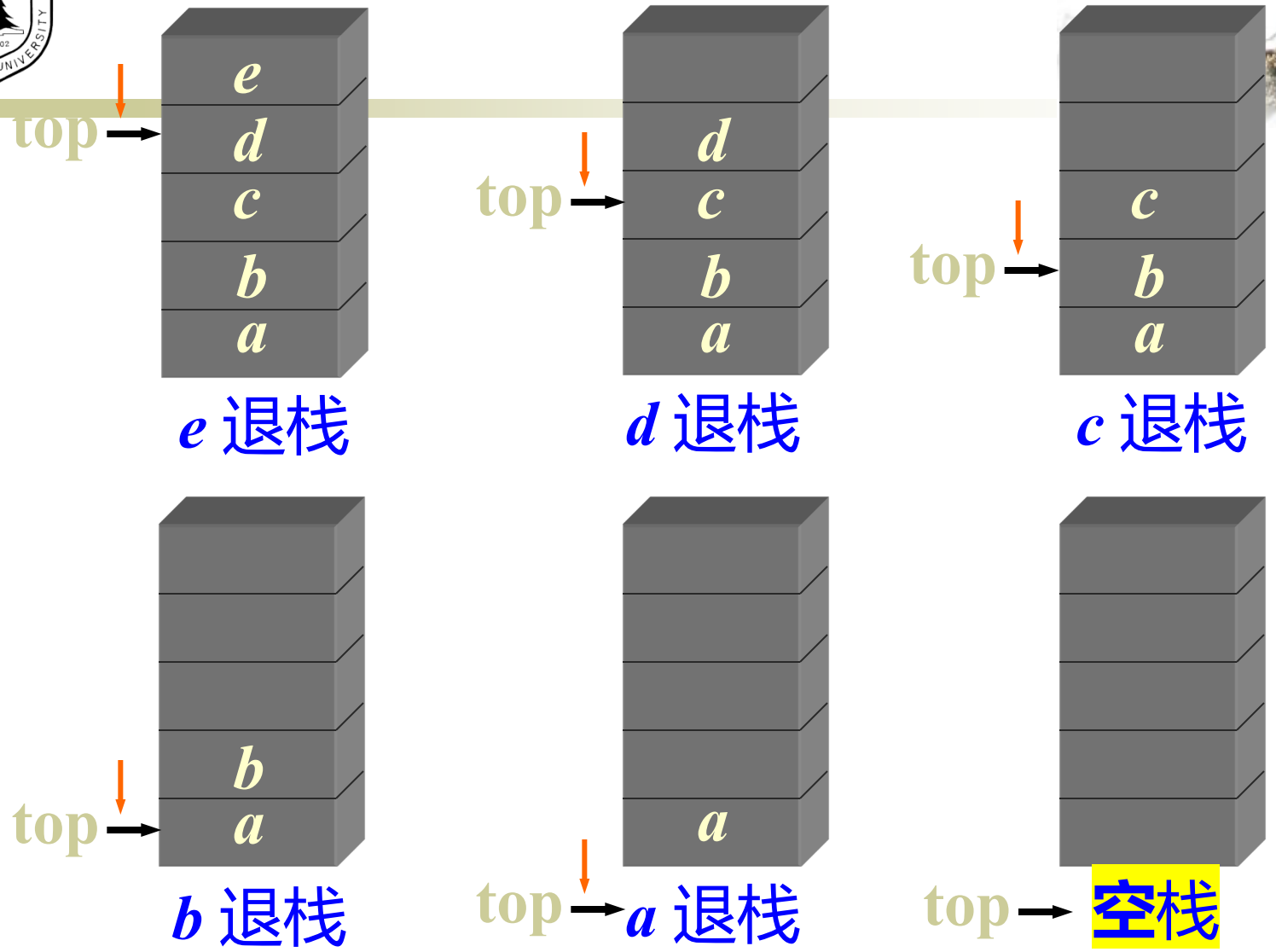


e 进栈



f 进栈溢出

进栈示例


top
e
d
c
b
a
e 退栈
top
d
c
b
a
d 退栈
top
c
b
a
c 退栈
top
b
a
b 退栈
top
a
a 退栈
top
空栈

退栈示例

顺序栈的操作

```
template <class E>
void SeqStack<E>::overflowProcess() {
//私有函数：当栈满则执行扩充栈存储空间处理
    E *newArray = new E[2*maxSize];
                    //创建更大的存储数组
     for (int i = 0; i <= top; i++)
         newArray[i] = elements[i];
     maxSize += maxSize;
    delete [ ]elements;
    elements = newArray;          //改变elements指针
};
```

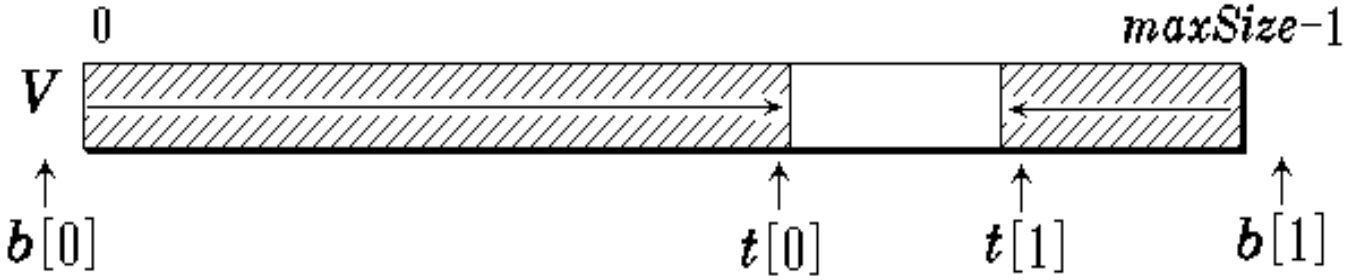
```
template <class E>
void SeqStack<E>::Push(E x) {
//若栈未满, 则将元素x插入该栈栈顶, 否则溢出处理
    if (IsFull() == true) overflowProcess();      //栈满
    elements[++top] = x;     //栈顶指针先加1, 再进栈
};

template <class E>
bool SeqStack<E>::Pop(E& x) {
//函数退出栈顶元素并返回栈顶元素的值
    if (IsEmpty() == true) return false;
    x = elements[top--];          //栈顶指针退1
   return true;                   //退栈成功
};
```

```
template <class E>
bool Seqstack<E>::getTop(E& x) {
//若栈不空则函数返回该栈栈顶元素的地址
    if (IsEmpty() == true) return false;
    x = elements[top];
  return true;
};
```

如何合理进行栈空间分配，以避免栈溢出或空间的浪费?

- 双栈共享一个栈空间（多栈共享栈空间）
- 栈的链接存储方式—— 链式栈

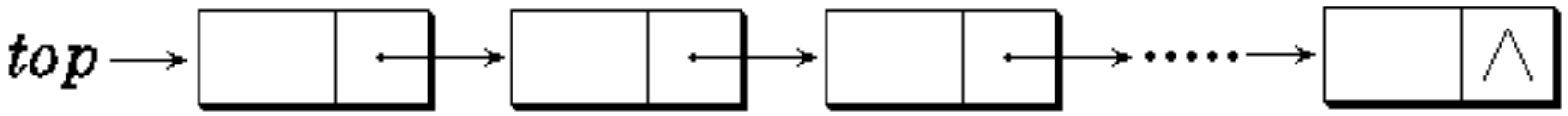
双栈共享一个栈空间



- 两个栈共享一个数组空间V[maxSize]
- 设立栈顶指针数组 t[2] 和栈底指针数组 b[2]
  t[i]和b[i]分别指示第 i 个栈的栈顶与栈底
- 初始 t[0] = b[0] = -1, t[1] = b[1] = maxSize
- 栈满条件：t[0]+1 == t[1] //栈顶指针相遇
- 栈空条件：t[0] = b[0]或t[1] = b[1]
  //栈顶指针退到栈底

```
bool Push(DualStack& DS, E x, int i) {
    if (DS.t[0]+1 == DS.t[1]) return false;
    if (i == 0) DS.t[0]++; else DS.t[1]--;
    DS.V[DS.t[i]] = x;
    return true;
}

bool Pop(DualStack& DS, E & x, int i) {
    if (DS.t[i] == DS.b[i]) return false;
    x = DS.V[DS.t[i]];
    if (i == 0) DS.t[0]--; else DS.t[1]++;
    return true;
}
```

3.1.3 链式栈（栈的链接表示）



- 链式栈无栈满问题，空间可扩充
- 插入与删除仅在栈顶处执行
- 链式栈的栈顶在链头

链式栈 (LinkedStack) 类的定义

```
#include <iostream.h>
#include "stack.h"
template <class E>
struct StackNode {                          //栈结点类定义
private:
    E data;                                 //栈结点数据
    StackNode<E> *link;                     //结点链指针
public:
    StackNode(E d = 0, StackNode<E> *next = NULL)
        : data(d), link(next) { }
};
```

```
template <class E>
class LinkedStack : public Stack<E> { //链式栈类定义
private:
    StackNode<E> *top;                    //栈顶指针

public:
    LinkedStack() : top(NULL) {}          //构造函数
    ～LinkedStack() { makeEmpty(); }      //析构函数
    void Push(E x);                       //进栈
    bool Pop(E& x);                       //退栈
```

```
bool getTop(E& x) const;              //取栈顶
bool IsEmpty() const { return top == NULL; }
void makeEmpty();                     //清空栈的内容
 friend ostream& operator << (ostream& os,
    LinkedStack<E> & s) ;
                                  //输出栈元素的重载操作 <<
};
```

链式栈类操作的实现

```
template <class E>
LinkedStack<E>::makeEmpty() {
                    //逐次删去链式栈中的元素直至栈顶指针为空
   StackNode<E> *p;
    while (top != NULL)                    //逐个结点释放
      { p = top;  top = top->link;  delete p; }
};

template <class E>
void LinkedStack<E>::Push(E x) {
                          //将元素值x插入到链式栈的栈顶, 即链头
   top = new StackNode<E> (x, top);      //创建新结点
    assert (top != NULL);                 //创建失败退出
};
```

```
template <class E>
bool LinkedStack<E>::Pop(E& x) {
                //删除栈顶结点, 返回被删栈顶元素的值
    if (IsEmpty() == true) return false;   //栈空返回
      StackNode<E> *p = top;                //暂存栈顶元素
      top = top->link;                      //退栈顶指针
      x = p->data;
    delete p;                               //释放结点
    return true;
};
```

```
template <class E>
bool LinkedStack<E>::getTop(E& x) const {
      if (IsEmpty() == true) return false;   //栈空返回
      x = top->data;                         //返回栈顶元素的值
    return true;
    };
```

```
template <class E>
ostream& operator << (ostream& os,
        LinkedStack<E> & s) {
                                //输出栈中元素的重载操作<<
    os << "栈中元素个数="<<s.getSize() << endl;
    LinkNode<E> * p = s.top; int i = 0;
    while (p != NULL) {
        os << ++i << ":" <<p->data << endl;
            p = p-> link;
       }
     return os;
};
```

思考：当进栈元素的编号为1, 2, …, n时，可能的出栈序列有多少种?

3.1.5 栈的应用：表达式的计算

- 算术表达式有三种表示：
  - 中缀(infix)表示

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$
$$③\quad ②\quad ①\quad ④\quad ⑦\quad ⑥\quad ⑤$$

    <操作数> <操作符> <操作数>，如 A+B；
  - 前缀(prefix)表示

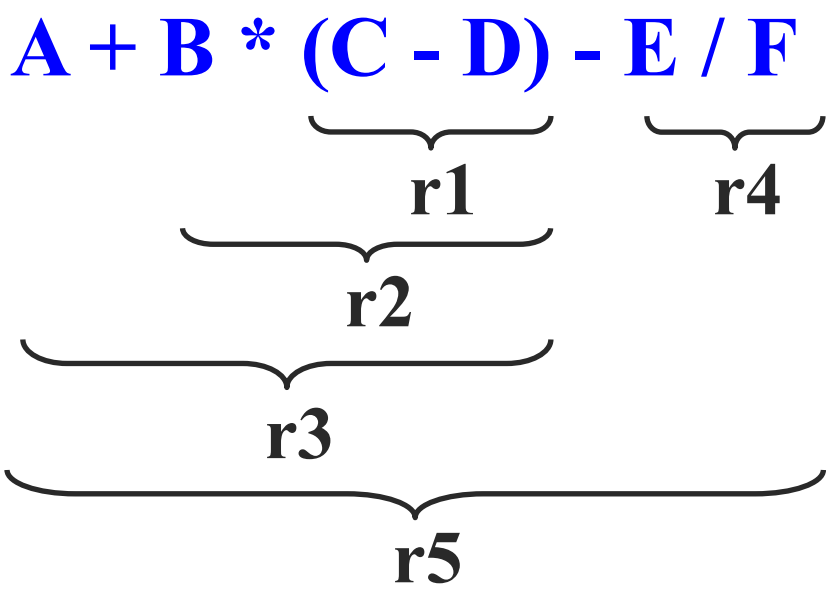
    <操作符> <操作数> <操作数>，如 +AB；
  - 后缀(postfix)表示

    <操作数> <操作数> <操作符>，如 AB+；

- **一般表达式的操作符有4种类型：**

1. **算术操作符** 如双目操作符（+、 –、 *、 / 和 %）以及单目操作符（-）。
2. **关系操作符** 包括<、<=、 ==、 !=、 >=、 >。这些操作符主要用于比较。
3. **逻辑操作符** 如与(&&)、或(||)、非(!)。
4. **括号** ‘(’和 ‘)’, 其作用是改变运算顺序。

表达式示例



中缀表达式 → **后缀表达式**

A + B * (C - D) - E / F　　ABCD-*+EF/-

中缀表达式　A + B * (C - D) - E / F

- 表达式中相邻两个操作符的计算次序为：
  - 优先级高的先计算
  - 优先级相同的自左向右计算
  - 当使用括号时从最内层括号开始计算

后缀表达式　A B C D - * + E F / -

- 在后缀表达式的计算顺序中已隐含了加括号的优先次序，括号在后缀表达式中不出现。

应用后缀表示计算表达式的值

idea：

- 从左向右顺序地扫描表达式，并用一个栈暂存扫描到的操作数或计算结果。
- 扫描中
  - 遇操作数则压栈；
  - 遇操作符则从栈中退出两个操作数，计算后将结果压入栈；
- 最后计算结果在栈顶。

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$

③ ② ① ④ ⑦ ⑥ ⑤

中缀表达式

① ② ③ ④ ⑤ ⑥ ⑦

15 7 1 1 + - ÷ 3 × 2 1 1 + + -

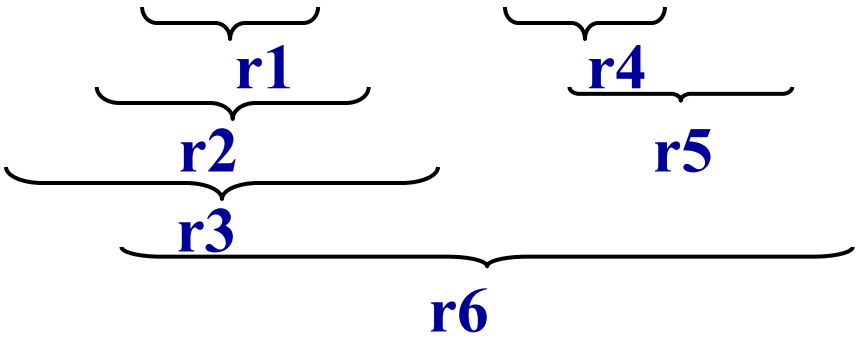
后缀表达式

后缀表达式的手算方法：

从左往右扫描，每遇到一个运算符，就让运算符前面最近的两个操作数执行对应运算，合体为一个操作数

注意：两个操作数的左右顺序

- 计算例 A B C D - * + E F ^ G / -



计算后缀表达式 A B C D - * + E F ^ G / -

步	输入	类 型	动 作	栈内容
1			置空栈	空
2	A	操作数	进栈	A
3	B	操作数	进栈	AB
4	C	操作数	进栈	ABC
5	D	操作数	进栈	ABCD
6	/	操作符	D、C 退栈，计算 C/D，结果 r1 进栈	ABr1
7	*	操作符	r1、B 退栈，计算 B*r1，结果 r2 进栈	Ar2
8	+	操作符	r2、A 退栈，计算 A+r2，结果 r3 进栈	r3

计算后缀表达式 A B C D - * + E F ^ G / -

步	输入	类 型	动 作	栈内容
9	E	操作数	进栈	r3E
10	F	操作数	进栈	r3EF
11	^	操作符	F、E 退栈，计算 E^F，结果 r4 进栈	r3r4
12	G	操作数	进栈	r3r4G
13	/	操作符	G、r4 退栈，计算 r4/G，结果 r5 进栈	r3r5
14	/"	操作符	r5、r3 退栈，计算 r34r5，结果 r6 进栈	r6

```
void Calculator :: Run ( ) {
   char ch;   double newoperand;
   while ( cin >> ch,  ch != ';' ) {
      switch ( ch ) {
         case '+' :  case '-' :   case '*' :   case '/' :
         case '^' :  DoOperator ( ch );  break;
                                          //计算
       default :  cin.putback ( ch );
                                          //将字符放回输入流
                  cin >> newoperand;  //读操作数
                  S.Push( newoperand );
      }
   }
}
```

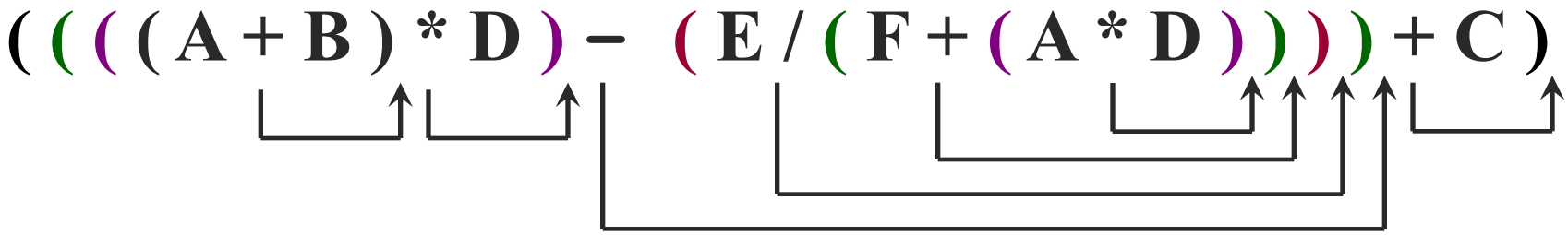
```
void Calculator :: DoOperator ( char op ) {
    //从栈S中取两个操作数，形成运算指令并计算进栈
  double left, right;   bool result;
  result = Get2Operands(left, right); //退出两个操作数
  if ( !result ) return;
  switch ( op ) {
    case '+': S.Push ( left + right);  break;     //加
    case '-': S.Push ( left - right);  break;     //减
    case '*': S.Push ( left * right);  break;    //乘
    case '/': if ( right != 0.0 ) {S.Push ( left / right); break; }
              else { cout << "除数为0!\n" );  exit(1); } //除
    case '^': S.Push ( Power(left,right) ); //乘幂
  }
}
```

```
bool Calculator :: Get2Operands(double& left, double& right ) {
//从栈S中取两个操作数
        if (S.IsEmpty( ) == true)
            {cerr << “缺少右操作数！ ”<< endl; return false;}
         S.Pop(right);

         if (S.IsEmpty( ) == true)
            {cerr << “缺少左操作数！ ”<< endl; return false;}
         S.Pop(left);

         return true;
}
```

中缀表示→后缀表示

- 先对中缀表达式按运算优先次序加上括号；
- 再把操作符后移到右括号的后面并以就近移动为原则；
- 最后将所有括号消去。

如中缀表示 (A+B)*D−E/(F+A*D)+C，其转换为后缀表达式的过程如下：



后缀表示　A B + D * E F A D * + / − C +

利用栈将中缀表示转换为后缀表示

- 使用栈可将表达式的中缀表示转换成它的后缀表示。
- 为了实现这种转换，需要考虑各操作符的优先级。

各个算术操作符的优先级

操作符 ch	#	(	*, /, %	+, −	)
isp (栈内)	0	1	5	3	6
icp (栈外)	0	6	4	2	1

- isp叫做栈内(in stack priority)优先级。
- icp叫做栈外(in coming priority)优先级。
- 操作符优先级相等的情况只出现在括号配对或栈底的”#”号与输入流最后的”#”号配对时。

中缀表达式转换为后缀表达式的算法

- 操作符栈初始化，将结束符‘#’进栈。然后读入中缀表达式字符流的首字符ch。
- 重复执行以下步骤，直到ch ＝ ‘#’，同时栈顶的操作符也是‘#’，停止循环。
  - 若ch是操作数直接输出，读入下一个字符ch。
  - 若ch是操作符，判断ch的优先级icp和位于栈顶的操作符op的优先级isp：

- 若 icp(ch) > isp(op)，令ch进栈，读入下一个字符ch。
- 若 icp(ch) < isp(op)，退栈并输出。
- 若 icp(ch) == isp(op)，退栈但不输出，若退出的是“(”号读入下一个字符ch。

- 算法结束，输出序列即为所需的后缀表达式。

步	输入	栈内容	语义	输出	动作
1		#			栈初始化
2	A	#		A	操作数A输出，读字符
3	+	#	+ > #		操作符+进栈，读字符
4	B	#+		B	操作数B输出，读字符
5	*	#+	* > +		操作符*进栈，读字符
6	(	#+*	(> *		操作符(进栈，读字符
7	C	# +*(		C	操作数C输出，读字符
8	-	# +*(	- > (		操作符-进栈，读字符
9	D	# +*(-		D	操作数D输出，读字符
10	)	# +*(-	) < -	–	操作符–退栈输出
11		#+*(	) = (		(退栈，消括号，读字符

步	输入	栈内容	语义	输出	动作
12	-	#+*	- < *	*	操作符*退栈输出
13		#+	- < +	+	操作符+退栈输出
14		#	- > #		操作符-进栈，读字符
15	E	#-		E	操作数E输出，读字符
16	/	#-	/ > -		操作符/进栈，读字符
17	F	#-/		F	操作数F输出，读字符
18	#	#-/	#< /	/	操作符/退栈输出
19		#-	#< -	-	操作符-退栈输出
20		#	# = #		#配对，转换结束

知识回顾与重要考点

- **表达式求值问题**
  - 概念 — 运算符、操作数、界限符（DIY概念：左操作数/右操作数）
  - 三种表达式
    - 中缀表达式 — 运算符在操作数中间
    - 后缀表达式（逆波兰式） — 运算符在操作数后面
    - 前缀表达式（波兰式） — 运算符在操作数前面

    一个中缀表达式可以对应多个后缀、前缀表达式
  - 后缀表达式考点
    - 中缀转后缀
      - ①按"左优先"原则确定运算符的运算次序
      - ②根据①中确定的次序，依次将各个运算符和与之相邻的两个操作数按<左操作数 右操作数 运算符> 的规则合体
    - 后缀转中缀 — 从左往右扫描，每遇到一个运算符，就将 <左操作数 右操作数 运算符> 变为 (左操作数 运算符 右操作数) 的形式
    - 计算 — 从左往右扫描，遇到操作数入栈，遇到运算符则弹出两个栈顶元素运算后入栈（注意：先弹出的元素是"右操作数"）
  - 前缀表达式
    - 中缀转前缀
      - ①按"右优先"原则确定运算符的运算次序
      - ②根据①中确定的次序，依次将各个运算符和与之相邻的两个操作数按 <运算符 左操作数 右操作数> 的规则合体
    - 计算 — 从右往左扫描，遇到操作数入栈，遇到运算符则弹出两个栈顶元素运算后入栈（注意：先弹出的元素是"左操作数"）